

EXERCÍCIOS DE APLICAÇÃO

1. Neste exercício, o objetivo é modelar um servidor alvo garantindo que a sua configuração inicial é válida e que não existem dados em falta.

- **a)** Crie uma classe denominada `TargetServer`. O construtor deve receber dois parâmetros: `ip_address` (str) e `os_version` (str | None = None).
- **b)** Defina uma constante de classe privada denominada `__DEFAULT_OS` com o valor "Unknown". No construtor, se o parâmetro `os_version` não for fornecido (ou seja, for None), o atributo da instância deve ser preenchido utilizando esta constante.
- **c)** O endereço IP é um dado crítico. Altere o construtor para que o IP seja guardado num atributo privado (`__ip_address`).
- **d)** Implemente os canais de comunicação utilizando o decorador `@property` para leitura e o `@ip_address.setter` para escrita. No `setter`, levante um `ValueError` se a *string* fornecida for vazia ou tiver menos de 7 caracteres.
- **e)** Na função `main()`, instancie dois objetos: um fornecendo o sistema operativo e outro omitindo-o. Tente alterar o IP de um deles para uma *string* inválida, capturando a anomalia com um `try-except`.

2. O objetivo é conjugar a orientação a objetos com o conceito de propagação de exceções (*Exception Bubbling*) e garantir que uma ligação de rede simulada é sempre encerrada de forma segura.

- **a)** Crie uma classe `AuthSession` cujo construtor exija o parâmetro `target_ip` (str). O construtor deve inicializar a variável passada e também definir um atributo de instância público `self.is_connected = False`.
- **b)** Desenvolva um método `execute_login(self) -> None`. Este método deve começar por alterar o estado para `self.is_connected = True` e imprimir "A tentar estabelecer ligação ao alvo...". Logo na linha seguinte, para simular uma defesa ativa, levante intencionalmente um erro utilizando `raise ConnectionError("Acesso rejeitado pela firewall")`.
- **c)** Aplique um bloco `finally` dentro do método `execute_login`. Este bloco deve garantir que o estado reverte para `self.is_connected = False` e imprimir "Socket de rede

destruído. Recursos libertados". É estritamente proibido utilizar um bloco try-except dentro deste método. O erro tem de se propagar para a main.

- **d)** Na função `main()`, instancie o objeto `AuthSession`. Invoque o método `execute_login()` dentro de um bloco try-except. Capture especificamente o `ConnectionError`, imprima o alerta no ecrã de forma limpa (sem o *Stack Trace*) e encerre o processo com `sys.exit(1)`.

3. Construa um gestor de vulnerabilidades em memória. Este exercício exige a aplicação de encapsulamento, tratamento de exceções contínuo e orquestração baseada em CLI.

- **a)** Crie a classe `Vulnerability`. O construtor deve receber `cve_id` (str) e `severity` (int).
- **b)** Encapsule a `severity` num atributo privado. Crie os validadores através de `@property`. O `setter` deve obrigar a que a severidade seja um número estritamente entre 1 e 10. Se for introduzido um valor fora do limite, levante um `ValueError`.
- **c)** Crie uma classe `ScannerSession`. Esta classe não recebe parâmetros no construtor. Deve inicializar um atributo de instância `self.found_vulns` como uma lista vazia (garantindo que não partilha memória). Adicione um método `add_vulnerability(self, vuln: Vulnerability)` para inserir objetos na lista.
- **d)** Na função `main()`, instancie um objeto `ScannerSession`. Crie um ciclo infinito (`while True`) que imprima o seguinte menu no ecrã:
 1. Registrar nova vulnerabilidade
 2. Listar vulnerabilidades registadas
 3. Encerrar sistema
- **e)** Trate o *input* do utilizador. Se o utilizador escolher a opção 1, peça o CVE e a severidade. Envolve esta captura num bloco try-except que intercete `ValueError`. Isto servirá para apanhar duas anomalias distintas: ou o utilizador digitou letras quando devia digitar o número da severidade, ou o `setter` da classe `Vulnerability` rejeitou o valor por não estar entre 1 e 10. Imprima o erro sem deixar o *script* ir abaixo, fazendo com que o menu volte a aparecer.
- **f)** Se o utilizador escolher a opção 3, invoque `sys.exit(0)` para fechar a ferramenta limpidamente.

